

Reflection: Building a 4-Agent Research Crew

UCLA Extension · Agentic AI · Week 1 Assignment · Ying Chen

1. How I designed the agents and why

I kept the design simple: four agents in a sequential crew, each with exactly one job, chained with context so every agent sees the outputs of the agents before it.

- Query agent (query_expander.py): breaks the main research question into several sub-questions and writes search queries for each one, using the papers' own key terms.
- Resource agent (source_hunter.py): the only agent with a tool. It uses search_papers to collect relevant passages from the 15-paper corpus and records each one verbatim with its paper id.
- Fact-Check agent (synthesizer.py): works only on what the Resource agent collected. It splits the material into themes, compares the papers' positions, and identifies where they agree and where they differ. It has no tools, so it cannot add facts that were never retrieved.
- Reporter agent (report_writer.py): writes the report in the required 7-section format and converts every paper id into an (Author, Year) citation, using a lookup table I generate from paper_index.json.

The key lesson I applied from the Week 1 lab: requirements work better as a checklist in each task's expected_output than as hints in the backstory. In my Exercise 2/3 experiments, a critic whose backstory said "watch for copied package names" still answered ACCURATE, but the same rules written as a mandatory PASS/FAIL checklist flipped the verdict to INACCURATE.

2. ReAct-style reasoning in the agent output

The Resource agent clearly follows a Thought - Action - Observation loop.

From my first run on "How do LLM agents improve their own reasoning?":

Action: search_research_papers('mechanisms of reasoning enhancement in large language models', k=5)

Observation: Error loading vector store: Could not connect to tenant default_tenant

Action: search_research_papers('strategies for improving reasoning in LLM agents', k=5)

Observation: [react_2023] "...outperforms IM-style prompting (ReAct-IM) (71 vs. 53 overall success rate)..." After a failed observation the agent did not stop; it reformulated the query and continued, and the second observation (ReAct's own 71

vs. 53 result) became evidence in the final report. In the second round of each run it issued the alternative queries planned by the Query agent (e.g. 'performance evaluation of reasoning methods', 'external tool integration effects on reasoning'), which surfaced Valmeekam et al. and Chain-of-Thought passages that the first round had missed.

The pipeline as a whole is also a ReAct structure: plan (reason) - retrieve (act) - synthesize over observations.

3. **What worked, what didn't, and what I changed**

My first version ran end-to-end but the reports were weak.

I diagnosed the problems from the logs and fixed them in v2. The original v1 reports are kept in the repository for comparison:

<https://github.com/MyTeachers123/agents-assignment-1/tree/main/outputs/v1>

a. Failed searches. About half of v1's searches returned 'Could not connect to tenant default_tenant', because CrewAI runs tool calls in parallel and they raced to initialise ChromaDB. I now open the collection once in crew.py before kickoff(); v2 had 0 failed searches.

b. Fabricated quotes. The tool truncates passages with '...', and the model 'completed' the sentences itself (e.g. ReAct quoted on 'inter-agent communication'). I added a rule to copy only visible text and keep '...', and switched the Resource agent to gpt-4o.

c. Wrong attribution. v1 described Wooldridge (1995) as discussing LLMs and labelled a Wang passage as Xi. The Fact-Check agent must now cite 'E# + paper id', with a rule against anachronisms.

d. Padded references. One v1 report listed 13 papers but cited about 5. Code now rebuilds the References section from the citations actually used in the text, so the list always matches.

e. Narrow evidence. Generic queries ('mechanisms of reasoning') returned mostly the two surveys. Giving the Query agent each paper's key concepts raised coverage to 6-10 papers per report. What still doesn't work.

The synthesis step can still over-reach: one report links Constitutional AI to a 'planning debate' and mentions 'transfer learning', neither of which comes from the retrieved evidence.

Prompt rules reduce hallucination but do not eliminate it; only code-level checks (like my reference rebuild) give guarantees. The provided chunks are also a limit: every chunk is labelled 'Full Text' and the tool shows only 500 characters, so some retrieved passages are reference lists.

I also learned that infrastructure matters: my crew first hung forever because Windows Firewall silently dropped Python's 127.0.0.1 traffic used by asyncio, which I found by dumping stack traces with faulthandler and fixed with a loopback-only firewall rule.

What I would change next: add a fifth Citation Verifier agent (or a Task guardrail) that string-matches every quote against the vector store and sends failures back for revision.

4. What production use would require

- Verification layer: programmatic quote and citation checks, with a human review step before publishing.
- Better retrieval: section-aware chunking, hybrid keyword + vector search, re-ranking, and returning full passages; track retrieval quality with the provided evals (Recall@k, MRR) as a regression test.
- Reliability and cost: retries and timeouts on tool/LLM calls, caching of repeated queries, per-run token budgets, and model routing (cheap models for planning, strong models where faithfulness matters).
- Observability and security: structured logs of every Thought/Action/Observation, secrets in a vault rather than .env, pinned dependency versions, and monitoring for drift as the corpus grows.
- Domain fit: the same architecture could power a multilingual community-resource assistant, but it would need source freshness dates, language-specific retrieval, and clear 'not covered' answers instead of guesses.